

Taller

Introducción a la detección de objetos espaciales
con Python e inteligencia artificial

Ing. Alejandro Núñez Arroyo

Cronograma de actividades

Clase 1: 26 de abril

Actividad teórica:

- Introducción a Computer Vision
- Tareas de Computer Vision
- Flujo de trabajo en un proyecto de Computer Vision
- Datasets para la detección de objetos espaciales
- Preparación de datos

Actividad práctica:

- Introducción a Google Colab y Python
- Componentes de una imagen

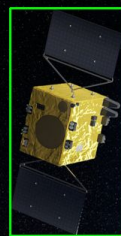
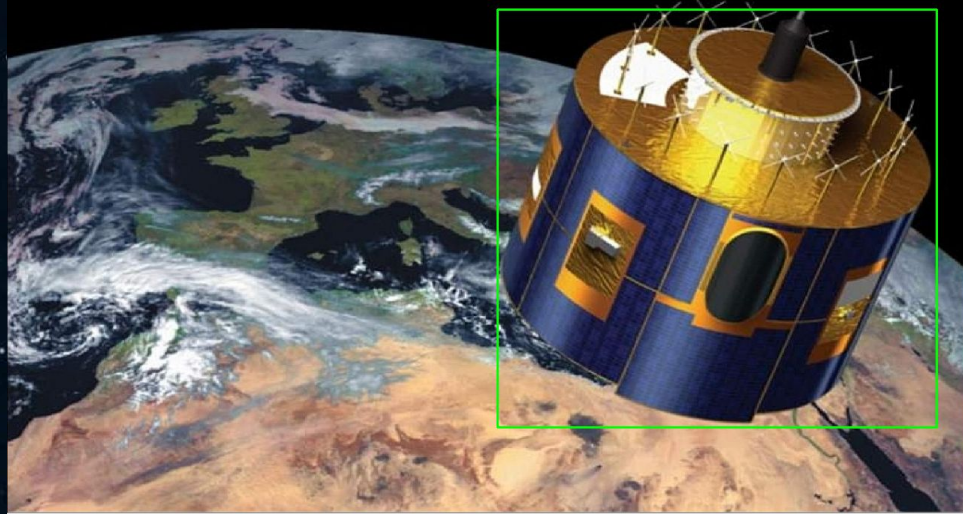
Clase 2: 3 de mayo

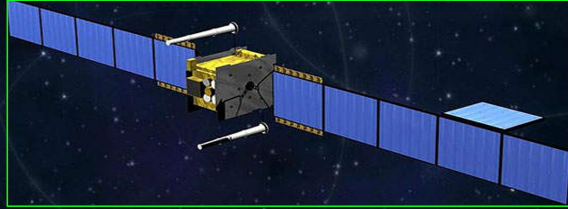
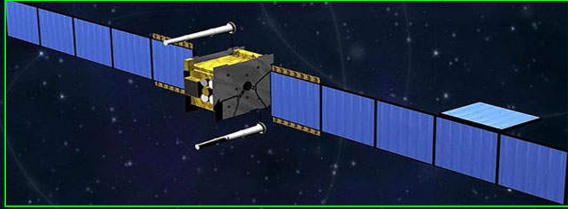
Actividad teórica:

- ¿Cómo funciona la detección de objetos?
- Modelos de clasificación de objetos (RF-DETR, YOLO, RetinaNet, etc)

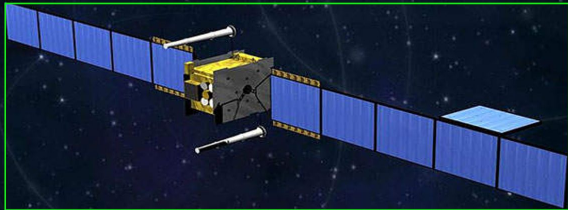
Actividad práctica:

- Preparación de datos
- Entrenamiento de modelos de clasificación de objetos espaciales
- Evaluación y consumo del modelo entrenado
- Conclusiones y trabajo a futuro





Clase 2



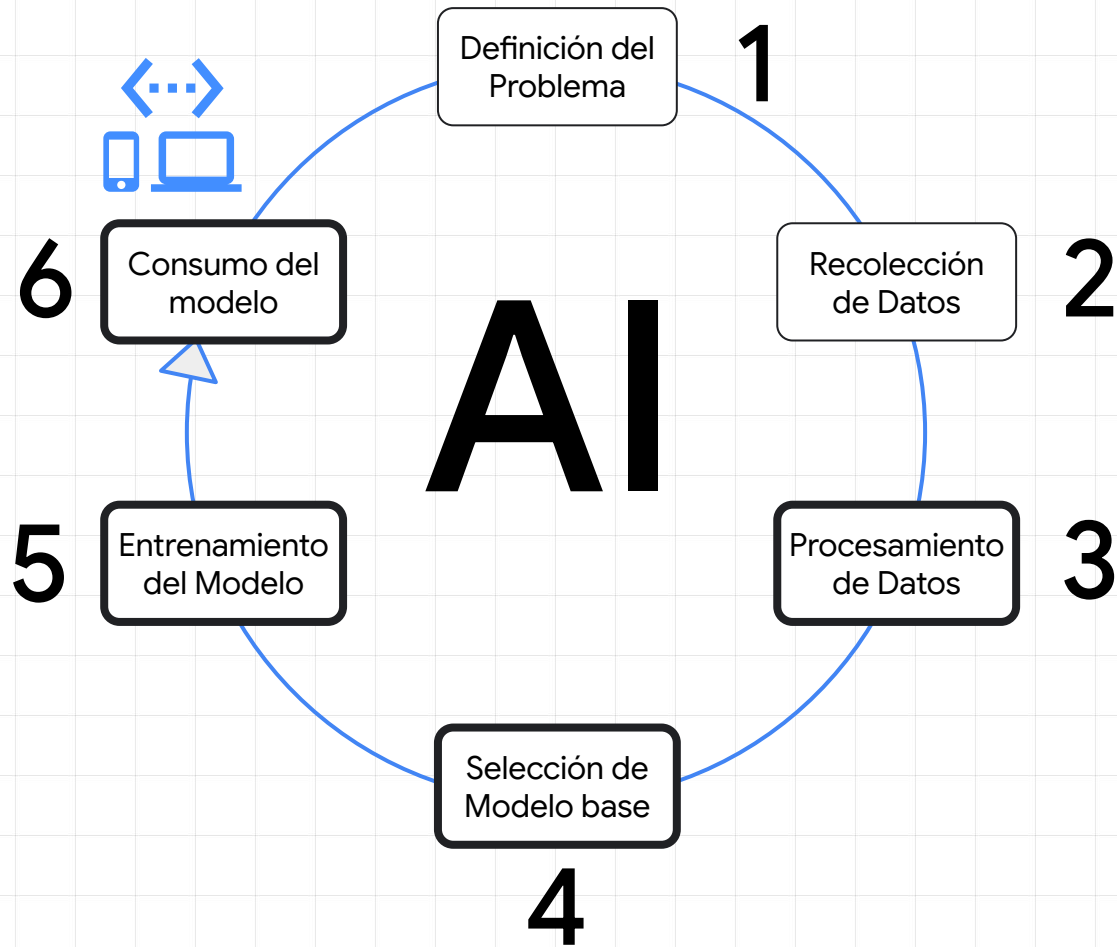


Imagen 403

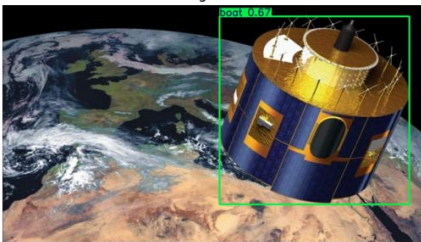


Imagen 419

Imagen 405

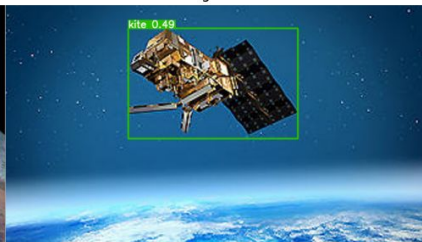


Imagen 423

Imagen 412

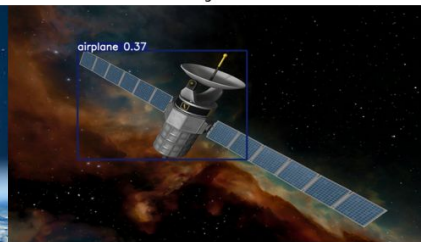


Imagen 429

Imagen 415

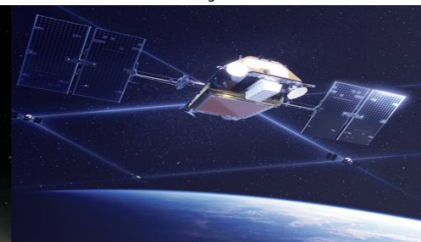


Imagen 430

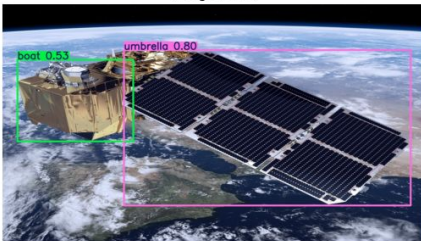


Imagen 403



Imagen 405



Imagen 412

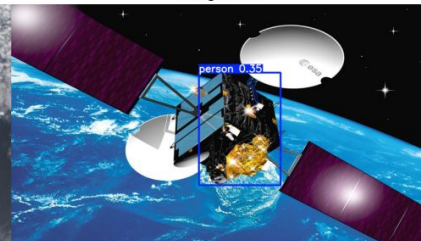


Imagen 415

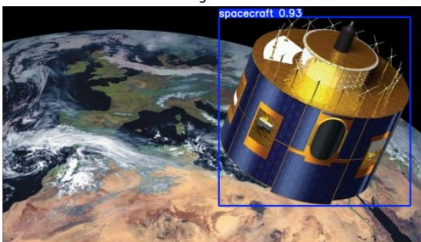


Imagen 419



Imagen 423

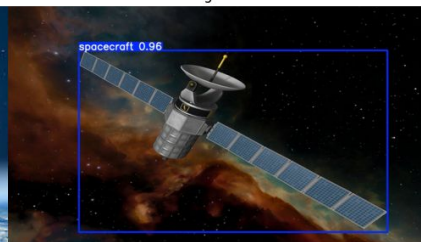


Imagen 429

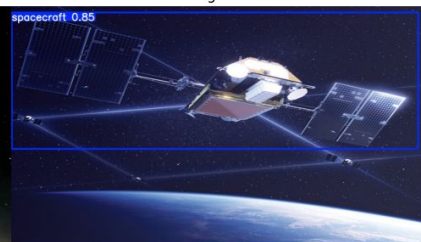
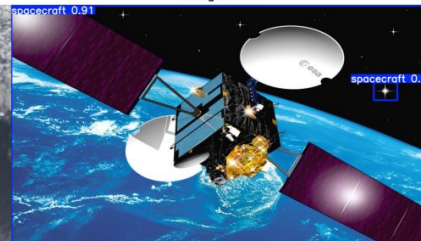
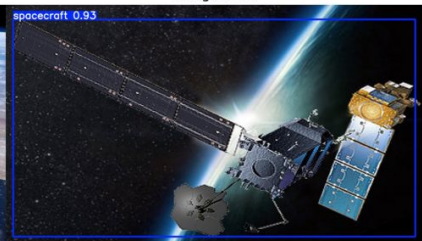
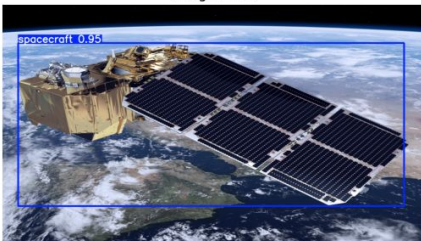


Imagen 430



¿Qué modelo usaremos?

Criterios para elegir un modelo de detección de objetos:

- **Experiencia técnica:** Capacidad para adaptar el modelo a la tarea específica.
- **Compatibilidad con recursos:** Asegurarse de que sea compatible con las versiones de CUDA y otros recursos utilizados.
- **Compatibilidad del framework:** Que el framework sea compatible con el modelo que queremos entrenar.
- **Consumo de GPU:** Evaluar la eficiencia en el uso de recursos de GPU durante la implementación.
- **Comunidad:** Disponibilidad de soporte y contribuciones de la comunidad.

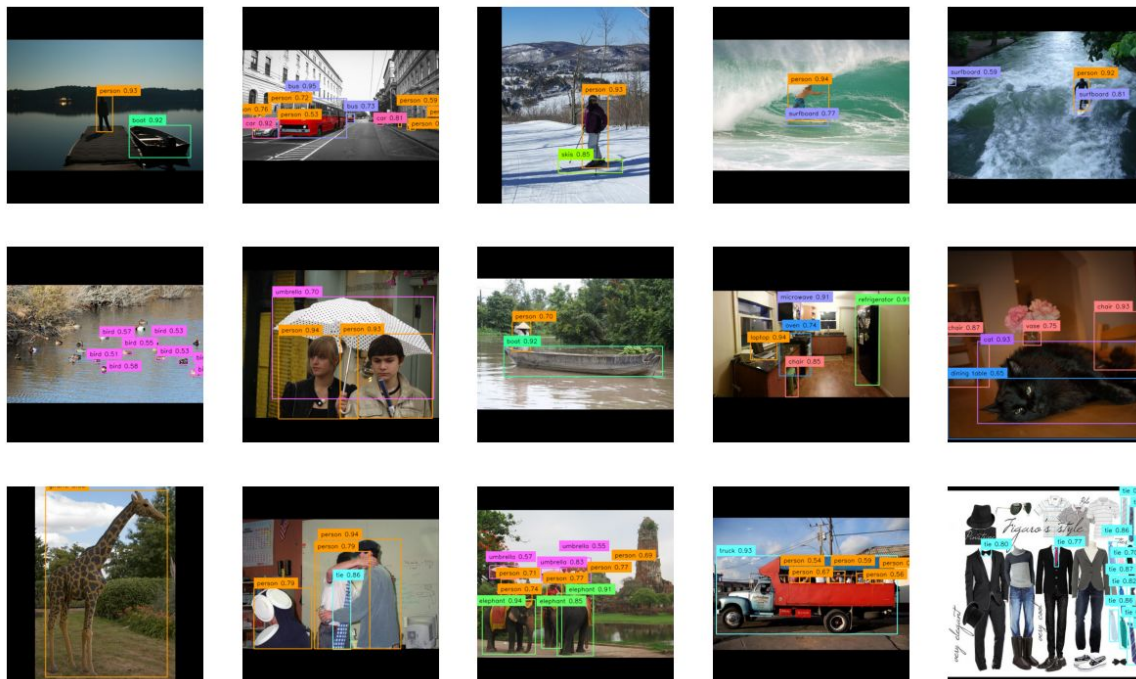
https://huggingface.co/models?pipeline_tag=object-detection&sort=trending

The screenshot shows the Hugging Face website interface. At the top, there's a search bar and navigation links for Models, Datasets, and Spaces. Below the navigation, there are tabs for Tasks, Libraries, Datasets, and Languages. The 'Tasks' tab is selected, and a filter for 'Object Detection' is applied. The page displays a list of trending models for Object Detection. The models listed are:

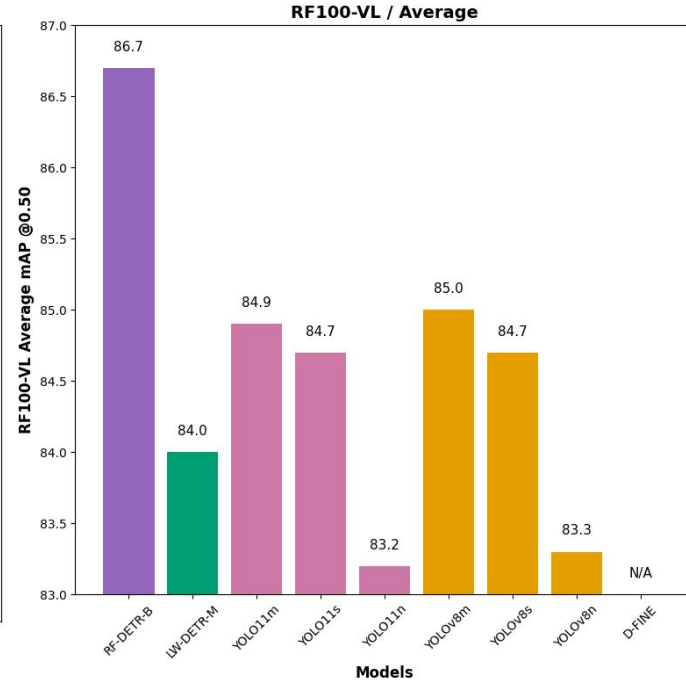
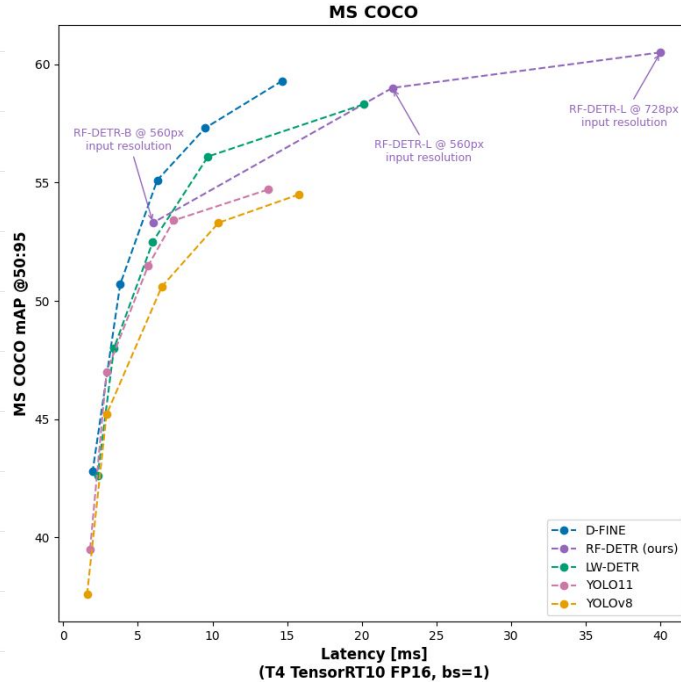
- Ultralytics/YOLOv8**: Object Detection, Updated Jan 11, 5.39k downloads, 215 likes.
- facebook/detr-resnet-50**: Object Detection, Updated Apr 10, 2024, 505k downloads, 859 likes.
- microsoft/conditional-detr-resnet-50**: Object Detection, Updated May 8, 2024, 5.24k downloads, 12 likes.
- unity/sentis-YOLOv8n**: Object Detection, Updated Mar 31, 49 downloads, 22 likes.
- tech4humans/yolov8s-signature-detector**: Object Detection, Updated 29 days ago, 28.8k downloads, 16 likes.
- jameslahm/yoloe**: Object Detection, Updated Mar 30, 39.7k downloads, 34 likes.

The left sidebar shows various task categories under 'Multimodal' and 'Computer Vision'. Under 'Multimodal', there are tasks like Audio-Text-to-Text, Image-Text-to-Text, Visual Question Answering, Document Question Answering, Video-Text-to-Text, Visual Document Retrieval, and Any-to-Any. Under 'Computer Vision', there are tasks like Depth Estimation, Image Classification, Object Detection (highlighted), Image Segmentation, Text-to-Image, Image-to-Text, Image-to-Image, Image-to-Video, and Unconditional Image Generation.

RF-DETR: SOTA Real-Time Object Detection Model



RF-DETR: SOTA Real-Time Object Detection Model



YOLO11



Unleashing Next-Gen AI

Discover more



Enhanced feature extraction architecture



Faster processing with balanced accuracy



Higher precision using 22% fewer parameters



Easily deployable on edge, cloud, and GPU systems



YOLO11 enterprise models providing enterprise-grade accuracy

Download the App



<https://docs.ultralytics.com/>

X

Y

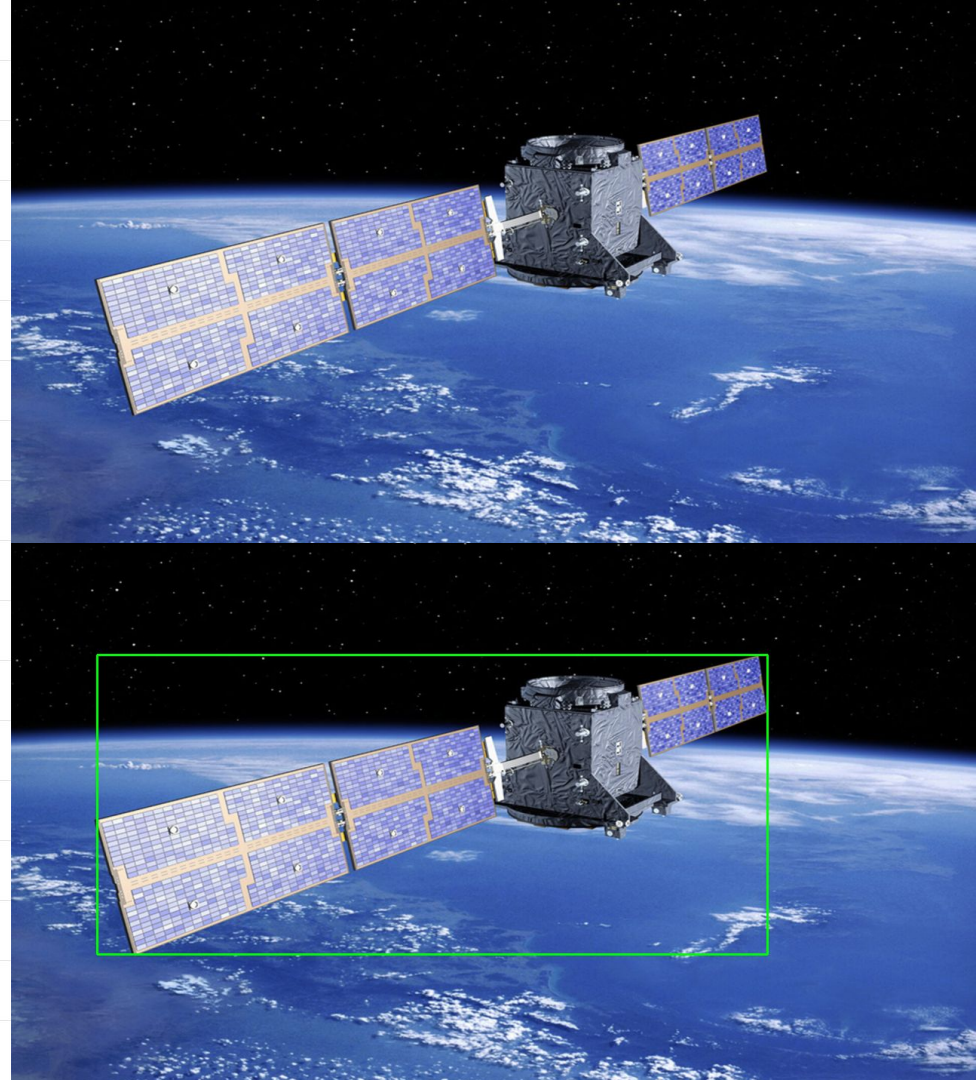


(0, 0) = 

¿Cómo se genera?

Cada modelo de detección de objetos **utiliza un formato específico** para las anotaciones, por lo que es importante asegurarse de que los datos etiquetados sean compatibles con el modelo que se desea entrenar. Los formatos de anotación más comunes incluyen:

- **YOLO (You Only Look Once):** Usa archivos de texto .txt por imagen, donde cada línea representa un objeto anotado con su clase y las coordenadas normalizadas del bounding box (x_center, y_center, width, height).
- **Pascal VOC:** Utiliza archivos XML por imagen, estructurados en un formato jerárquico que incluye metadatos de la imagen y las coordenadas de los objetos (xmin, ymin, xmax, ymax).
- **COCO (Common Objects in Context):** Almacena todas las anotaciones en un solo archivo JSON que contiene información sobre imágenes, categorías y anotaciones con bounding boxes y/o segmentaciones.

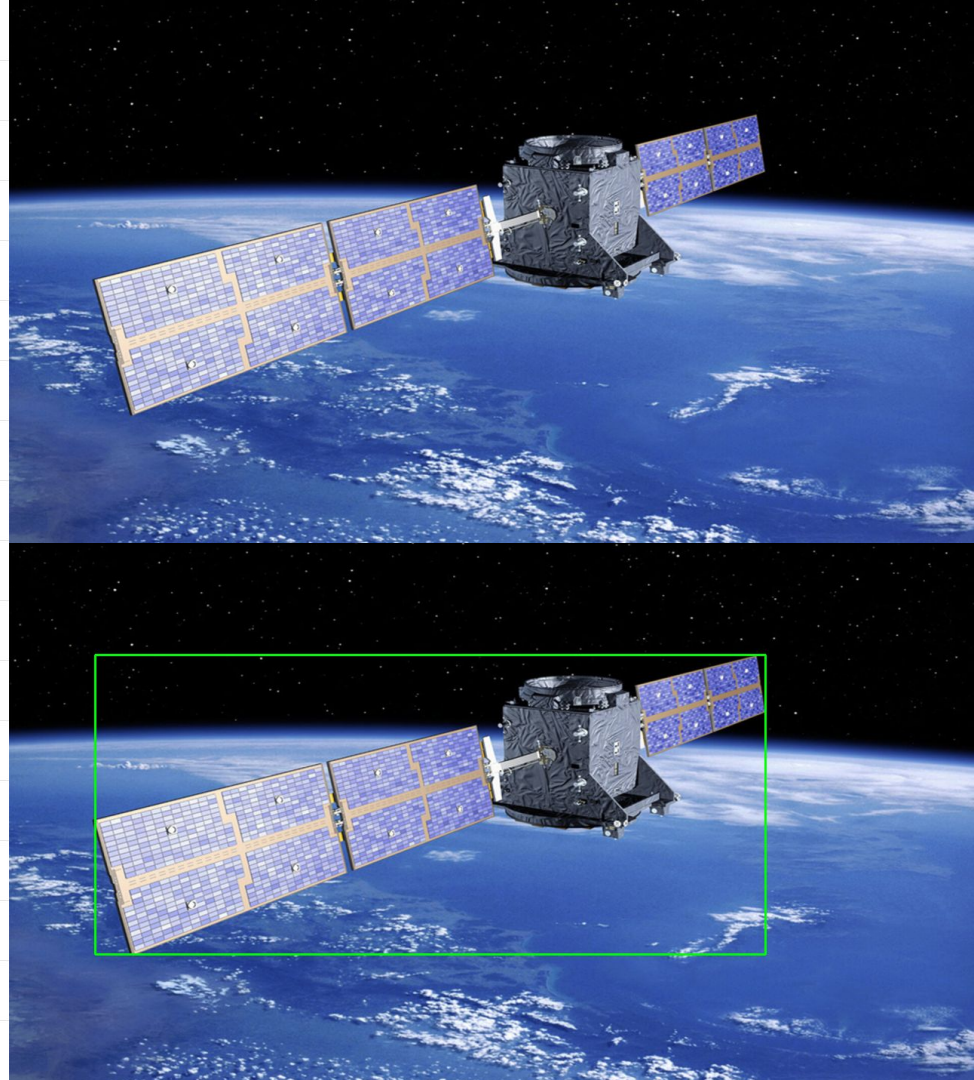


¿Cuál usaremos nosotros?

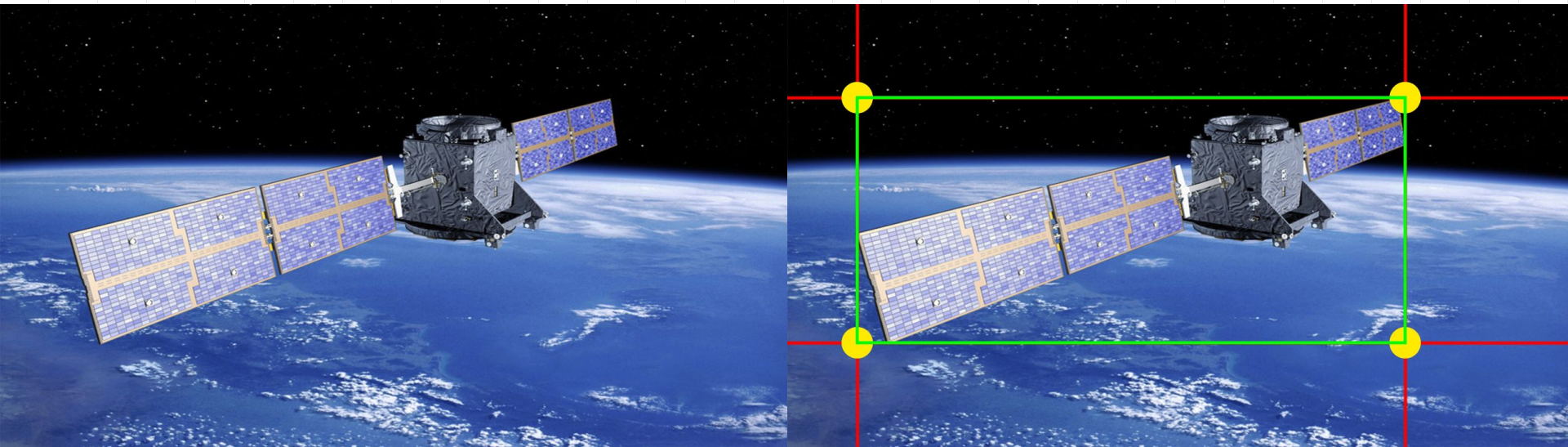
Entrenaremos con YOLO, entonces necesitamos:

- **YOLO (You Only Look Once):** Usa archivos de texto .txt por imagen, donde cada línea representa un objeto anotado con su clase y las coordenadas normalizadas del bounding box (x_center, y_center, width, height).

¿Nuestro dataset ya está en ese formato?



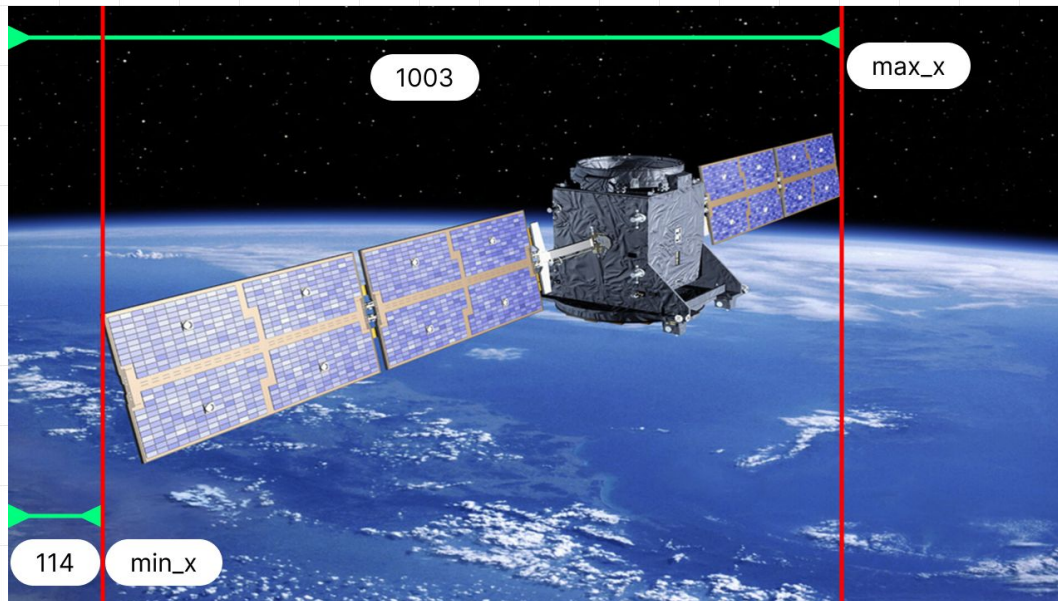
¿Cómo son nuestros datos?



“404”: `[[1003, 549, 114, 152]]`

“Número de Imagen”: `[[xmax, ymax, xmin, ymin]]`

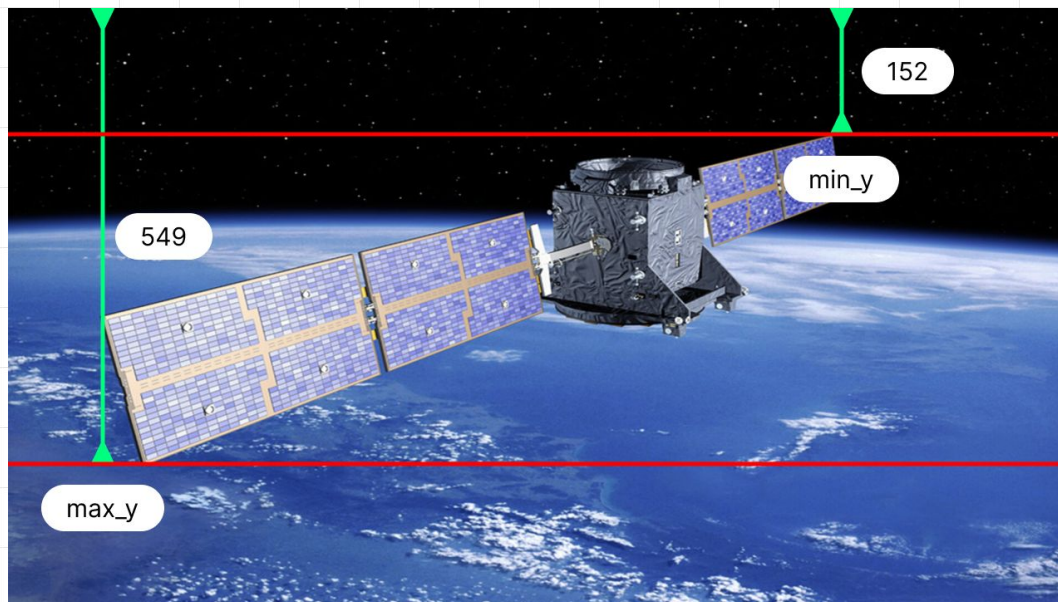
¿Cómo son nuestros datos?



“404”: [[1003, 549, 114, 152]]

“Número de Imagen”: [[xmax, ymax, xmin, ymin]]

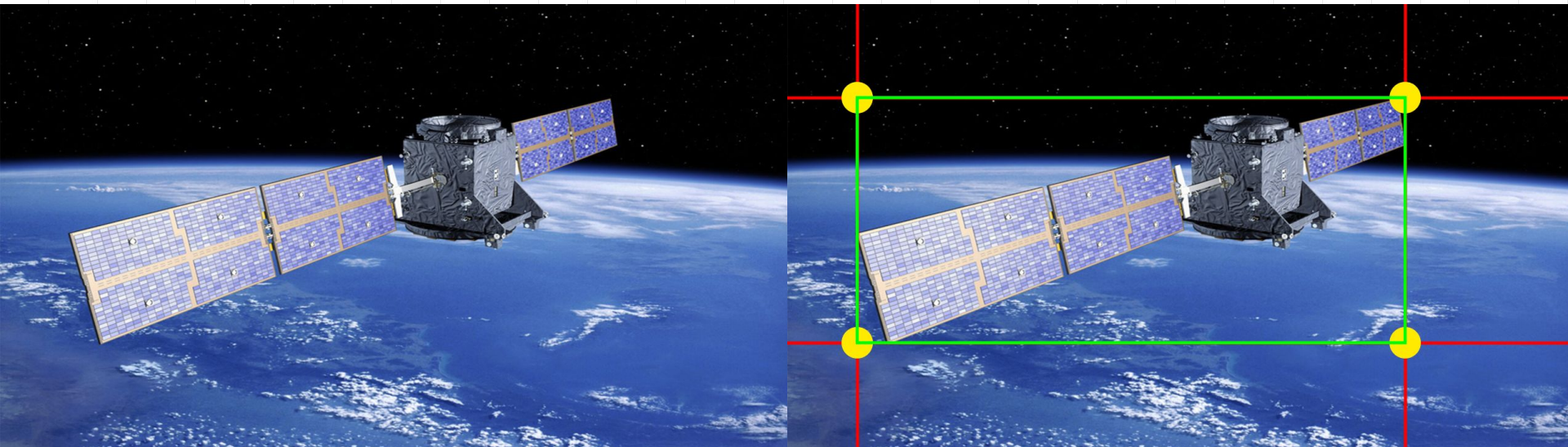
¿Cómo son nuestros datos?



“404”: [[1003, **549**, 114, **152**]]

“Número de Imagen”: [[xmax, ymax, xmin, ymin]]

¿Cómo son nuestros datos?

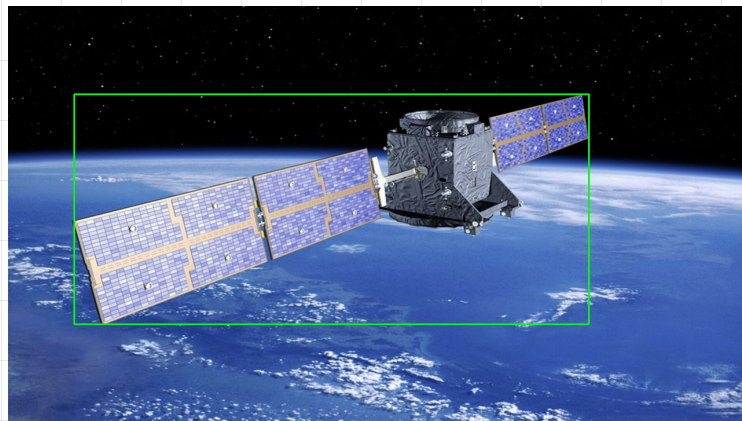


“404”: `[[1003, 549, 114, 152]]`

“Número de Imagen”: `[[xmax, ymax, xmin, ymin]]`

Conversión de formato

YOLO (You Only Look Once): Usa archivos de texto .txt por imagen, donde cada línea representa un objeto anotado con su clase y las coordenadas normalizadas del bounding box (x_center, y_center, width, height).



```
width, height = 1280, 720
bbox = [1003, 549, 114, 152]
max_x, max_y, min_x, min_y = bbox
x = min_x
y = min_y
w = max_x - min_x
h = max_y - min_y
```

```
x_center = (x + w / 2) / width
y_center = (y + h / 2) / height
w_norm = w / width
h_norm = h / height
```

Formato inicial:

“404”: [[1003, 549, 114, 152]]

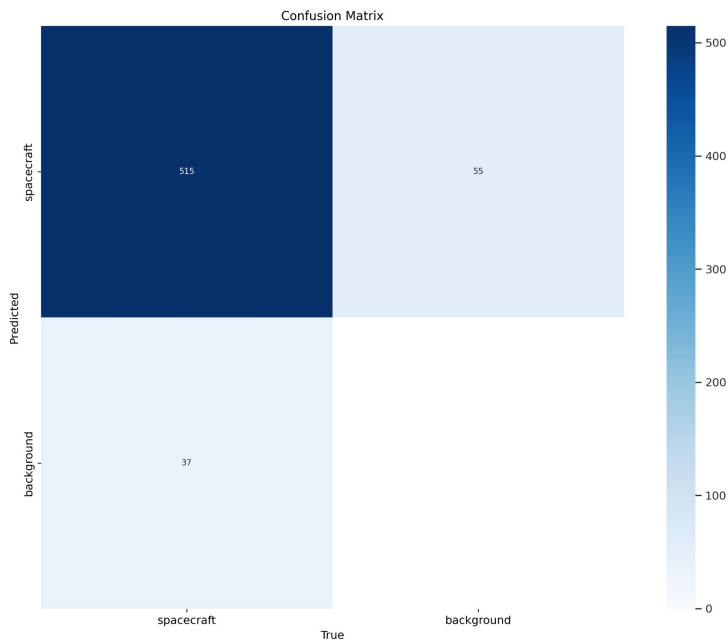
Formato YOLO:

0 0.436328 0.486806 0.694531 0.551389

objeto, x_center, y_center, width, height

Resultados - Matriz de confusión

Permite evaluar el desempeño de un modelo clasificando correctamente y erróneamente las predicciones de las clases del dataset.



Imágenes en validación: 444 imágenes

Cantidad de objetos: 552 objetos

- **Verdaderos Positivos (VP - 515):** Se predijo correctamente “spacecraft” cuando realmente era “spacecraft”.
- **Falsos Positivos (FP - 55):** Se predijo “spacecraft”, pero en realidad era “background”.
- **Falsos Negativos (FN - 37):** Se predijo “background”, pero en realidad era “spacecraft”.
- **Verdaderos Negativos (VN):** No aplica en este caso



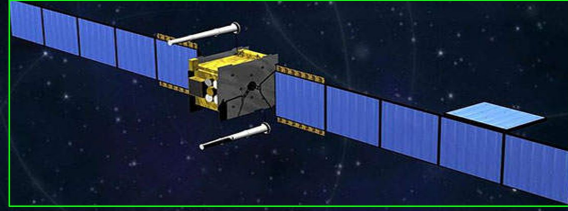
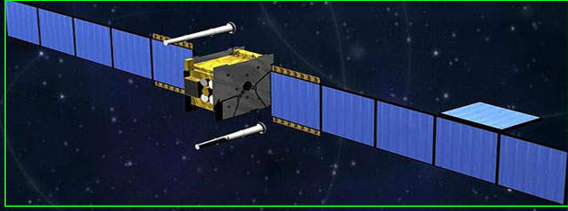
Sé parte del **hackatón** de Inteligencia Artificial más importante de Bolivia

Demuestra tu destreza y crea el futuro junto a otros apasionados por la tecnología

Inscribirme

Convocatoria

<https://gdgbolivia.com/>



Gracias
¿Preguntas?

